

Semantic Indexing for Messy and Decompiled Codebases

2026-06-21 • 22 Sources

22

SOURCES

8

VERIFIED CLAIMS

6

MAIN FINDINGS

4

INDEX LAYERS

EXECUTIVE SUMMARY

The strongest conclusion is that decompiled code should not be indexed as raw text alone. Decompiled outputs often preserve some control-flow, type, call, literal, and API-use information, but lose comments and meaningful identifiers; the decompiler literature treats identifier recovery as technically impossible in the exact sense, while still showing that usage context can predict useful names and types with meaningful accuracy [1], [2], [3], [4], [5]. For an MCP semantic-search tool, the practical lesson is to enrich each code chunk before embedding it.

The recommended architecture is a multi-representation index. Each function or logical block should store raw code, normalized code, a compact natural-language summary, inferred variable roles, literals/API tokens, structural metadata, and relationship edges such as callers, callees, Roblox instance paths, remotes, services, and event connections [8], [10], [12], [13], [14], [15]. Retrieval should then combine dense semantic search, sparse or BM25 keyword search, metadata filters, and graph expansion instead of relying on one vector score [17], [18], [19].

Generated summaries and inferred names are valuable, but they should be treated as retrieval annotations rather than truth. The report recommends a staged pipeline: parse and chunk, normalize placeholder identifiers, extract structural features, generate short summaries and role labels, index several searchable fields, retrieve with hybrid fusion, expand around graph neighbors, rerank with a code-aware model or lightweight LLM pass, and evaluate against a small task-specific query set [18], [20], [21], [22]. This should materially improve search quality for "what does

this code do?" queries over decompiled scripts where names like v1, v2, or a3 carry almost no meaning.

INTRODUCTION

The research question was: what is the best way to semantically index a messy codebase when the searchable corpus is decompiled code and variable names are not semantically useful? I interpreted "best" as practical retrieval quality for an MCP server, not as a new machine-learning research project. The target user is an LLM or coding agent asking questions such as "where is purchase validation handled?", "what remote fires inventory updates?", or "find the code that rate-limits teleport requests" against decompiled scripts.

The key assumption is that the corpus is closer to decompiled Roblox/Luau source than to clean application code. The exact academic decompiler papers mostly study C/C++ binaries and tools such as Hex-Rays or Ghidra, but the information-loss pattern transfers: original local names and comments are often unavailable, the remaining text is noisy, and many identifiers are placeholders [1], [2], [3], [5]. The implementation recommendations therefore focus on robust retrieval features that survive poor naming: structural chunking, call and data-flow clues, string literals, framework API names, constants, object paths, event/remoting relationships, and generated summaries.

The methodology combined targeted web retrieval, primary academic sources, implementation documentation, and system papers. Priority was given to sources that directly address decompiled variable recovery, code representation learning, semantic code search benchmarks, code chunking, hybrid retrieval, context retrieval for coding assistants, and incremental codebase indexing. The citations in this report are intentionally skewed toward papers and mature docs rather than vendor marketing.

MAIN ANALYSIS

Finding 1: Decompiled code requires enrichment because raw identifiers are degraded signals

The decompiler-specific literature is unusually clear: local identifier names are lost in compilation, and decompilers often replace them with generic placeholders, even when they recover useful structure [1], [2], [3]. Jaffe et al. describe decompiled variable names as meaningless placeholders and frame the recovery task as generating natural names from usage context rather than recovering guaranteed originals [1]. DIRE similarly states that decompilers can reconstruct structure and type information but do not reconstruct semantically meaningful variable names [2]. DIRTY extends that line by predicting both names and types from decompiler context, and it reports original-name recovery at 66.4 percent and original-type recovery at 75.8 percent on its dataset [3]. VarBERT reports a simpler constrained masked-

language-model approach with 84.15 percent exact original-name prediction on an existing large-scale dataset, although it is still a research setting, not a guarantee for arbitrary game decompilation [4]. STRIDE later shows that even token-sequence matching can be competitive with heavier transformer models for decompiled type and name prediction, which is important because it suggests that local token context remains useful even when raw names are weak [5].

For indexing, this means the search document should not simply be "the decompiled source text." A chunk containing `local v1 = v2[v3]` is nearly invisible to natural-language retrieval, but the same chunk may include API calls, literals, table keys, service names, event connections, remote names, numeric thresholds, path-like instance names, and control-flow patterns. Those features should be extracted and promoted into fields that embedding models and sparse retrieval can actually see. In a Roblox/Luau MCP server, high-value extracted fields would include `game:GetService(...)` names, `RemoteEvent` and `RemoteFunction` names, `FireServer/InvokeServer` call sites, `Connect` handlers, `WaitForChild` paths, string constants, enum names, metatable operations, module requires, returned table keys, and assignment targets.

The evidence also suggests a useful middle path. Perfect renaming is not needed for search. The goal is to create retrieval annotations that say "this variable behaves like player", "this function validates an item id", or "this block listens to an inventory remote." The research systems recover names or types by using context, lexical features, ASTs, and token sequences [2], [3], [5]. A production MCP index can use the same principle without training a full recovery model: generate short role labels and summaries from chunk-local evidence, keep them explicitly marked as inferred, and index them alongside the original code.

Finding 2: Program-aware chunking beats file-level or arbitrary text chunking

Semantic code search depends heavily on chunk boundaries. CodeSearchNet framed semantic code search as retrieving code from natural-language queries and provided benchmark infrastructure around function-level corpora from multiple languages [6]. Modern code-representation models also tend to operate on functions, methods, snippets, or structured program fragments rather than whole repositories [7], [8], [9], [10], [12], [13]. Qdrant's code-search tutorial makes the same practical point: functions, methods, structs, enums, and similar language-specific constructs are good chunk candidates because they are meaningful but still small enough for embedding models [14].

For messy or decompiled code, chunking should be more defensive than normal source-code chunking. The best primary boundary is a function or closure, because user questions usually map to behavior. The second boundary is a meaningful event or callback body, because decompiled Roblox code often hides behavior in anonymous closures passed to `Connect`, `spawn`, `task.defer`, promises, tween callbacks, remote handlers, or UI button events. The third boundary is a fallback sliding window for files that cannot be parsed cleanly. LlamaIndex's `CodeSplitter` uses AST nodes via tree-sitter for code documents [15], while LangChain's recursive splitter is a reasonable fallback that tries larger semantic separators before

smaller ones [16]. The practical recommendation is to prefer a Luau-aware parser when available, but to degrade gracefully into syntactic heuristics rather than refusing to index.

Each chunk should also carry parent and neighbor context. A callback chunk without its surrounding service setup or table declaration can be hard to interpret, while embedding the entire file introduces noise. The compromise is parent-summary metadata: file path, script name, Roblox hierarchy, enclosing function name if present, surrounding module table name, adjacent comment if any, and a short parent chunk summary. This keeps chunks small while giving the reranker enough clues to resolve ambiguous behavior.

Finding 3: A multi-representation index is stronger than one embedding per raw chunk

General code embedding work supports the idea that code has structure beyond flat token sequences. CodeBERT learns joint programming-language and natural-language representations and is evaluated on natural-language code search and documentation generation [7]. GraphCodeBERT improves on token-only representation by incorporating data flow, described as a semantic-level relation showing where a value comes from [8]. CodeT5 explicitly uses identifier-aware objectives and comments to improve code understanding and generation [9]. UniXcoder incorporates AST and comments, uses contrastive learning for code-fragment representation, and evaluates code search and code-to-code search [10]. Earlier code2vec and code2seq show the same broader pattern from another direction: AST paths can encode useful semantic properties and support tasks such as method-name prediction, summarization, documentation, and retrieval [12], [13].

The direct implication is that an index should store multiple views of the same chunk. Raw code is useful for exact syntax and code-model embeddings. Normalized code is useful when decompiled names add noise; it should replace low-information locals like `v1`, `v2`, `a3`, `arg_0`, or generated upvalue names with placeholders while preserving meaningful global, API, string, and property tokens. A "semantic card" is useful for natural-language queries: a compact generated description with fields such as purpose, inputs, outputs, side effects, calls, events, remotes, data stores, UI elements, suspicious checks, and inferred variable roles. A sparse token field is useful for exact lookup: strings, paths, constants, identifiers, API names, and table keys. A graph metadata field is useful for expansion: callers, callees, required modules, required-by modules, remotes used, instance path, service names, and neighboring callbacks.

This design reduces dependence on any single model. A general text embedding may understand "purchase validation" but miss `MarketplaceService` or a Roblox remote name. A code embedding may see loops and API calls but may not bridge from English user intent. Sparse search will catch exact names and literals that embeddings can miss. Graph expansion will find caller/callee context when the top vector hit is only a helper. This is especially important in decompiled code where identifier noise can dominate the token stream.

Finding 4: Hybrid retrieval plus reranking is the best practical baseline

The retrieval stack should be hybrid by default. Elastic describes hybrid search as combining lexical and semantic retrieval, with lexical search contributing precision for rare terms, structured queries, and domain-specific language, while semantic retrieval handles intent and related concepts [17]. Qdrant documents hybrid dense and sparse retrieval with reciprocal rank fusion, allowing sparse and dense candidates to be fused into one result set [18]. Sourcegraph's Cody context paper makes the same architectural point for coding assistants: retrieval sources should be complementary, and context selection must balance recall, precision, latency, and token budget [19].

For this MCP server, the initial candidate stage should retrieve broadly. One branch should run dense search over the generated semantic card. Another should run dense search over raw or normalized code with a code-oriented embedding model. A third should run sparse/BM25 search over exact tokens and metadata. If query latency allows it, a HyDE-style query-expansion step can ask a model to generate likely code terms or a hypothetical code snippet from the user's natural-language query, then search with that expanded query [21]. LanceDB's codebase RAG writeup uses similar ideas in practice: generated comments, hybrid search, HyDE, and reranking to bridge natural-language queries and code [22].

After candidate generation, reranking should be explicit. For small local result sets, an LLM or cross-encoder-style scoring pass can read the query, semantic card, metadata, and a bounded code excerpt, then score relevance and explain the match. The reranker should prefer chunks that satisfy the user intent, not merely chunks that have similar vocabulary. It should also deduplicate by function, file, and parent script so that the final context sent to the model is diverse. This matters for LLM performance: the output of semantic search is not the final answer, it is context packing. Bad retrieval floods the model with plausible but irrelevant code, increasing token load and hallucination risk.

Finding 5: Generated names and summaries are useful, but they must be labeled as inferred

Identifier recovery papers show that meaningful names can often be inferred from context, but they also show why overconfidence would be dangerous. Jaffe et al. emphasize that exact original names are impossible in principle once lost, even though natural names can be generated from context [1]. DIRE, DIRTY, VarBERT, and STRIDE report useful results, but they operate under benchmark assumptions and language/tooling setups that may not match a Luau decompiler corpus [2], [3], [4], [5].

The safest production pattern is to generate annotations without rewriting the canonical source. For example, store `inferred_roles`: `{"v12": "player", "v14": "itemId", "v18": "price"}` with confidence and evidence snippets such as "used in `Players:GetPlayerByUserId`", "compared against item table keys", or "passed to `MarketplaceService`". Store summary: "Validates an inventory purchase request, checks item price, and fires a UI update remote" separately from the raw chunk. Store `normalized_code` separately from `raw_code`. Search can use all of these, but user-facing output should show raw code plus the inferred annotations so an LLM can inspect the evidence rather than treat the annotation as ground truth.

This is also where decompiled-code indexing can outperform generic semantic indexing. Generic vector search over raw chunks might miss behavior when names are bad. A purpose-built enrichment pass can identify stable semantic anchors that survive decompilation: called functions, constants, string literals, service names, event names, table key names, arithmetic checks, branch predicates, return shape, and data-flow roles. GraphCodeBERT's data-flow emphasis supports this intuition at the model level [8], while code2vec/code2seq support using structural paths rather than tokens alone [12], [13].

Finding 6: Incremental indexing and evaluation are part of retrieval quality, not just performance polish

Indexing can be expensive, especially if it includes generated summaries. Cursor's codebase-indexing post states that embeddings are the expensive step, so it performs the work asynchronously and caches embeddings by chunk content [20]. That design maps well to this MCP server. A chunk should have a stable ID based on script identity, function boundary, normalized text hash, and parent metadata. If only one script changes, the system should update only changed chunks and preserve prior summaries and embeddings for unchanged chunks. This makes full-index semantics practical without returning stale partial results.

Evaluation is equally important. CodeSearchNet exists because semantic code search needed shared corpora, metrics, and benchmarks [6]. The Cody paper argues that context engines need retrieval and ranking evaluation, but code-assistant evaluation is hard because relevant workspace state can be ephemeral and labeled data is scarce [19]. For this MCP server, a small local eval set is enough to guide improvements. Create 50 to 100 query-answer pairs from known decompiled scripts: each query, expected script or function IDs, expected remote/module/service anchors, and a difficulty label. Track recall@5, recall@10, MRR, token cost, duplicate rate, and "answerable with returned context" judged by an LLM or manual review. Then ablate raw embeddings, normalized embeddings, generated summaries, BM25, graph expansion, HyDE, and reranking.

Without evaluation, semantic indexing will feel better when it returns fluent-looking matches but may quietly miss critical code. With evaluation, the team can tune chunk size, summary prompts, fusion weights, score thresholds, and reranker behavior against actual decompiled-code tasks.

SYNTHESIS & INSIGHTS

The best strategy is not one technique. It is a layered retrieval pipeline that accepts the weakness of decompiled identifiers and compensates with redundant signals. The cleanest mental model is "turn each chunk into a small evidence record." The raw source remains the audit trail. The normalized source removes decompiler naming noise. The extracted feature record exposes stable anchors. The generated semantic card bridges natural-language user queries to code behavior. The graph links place each chunk in its runtime context. Dense search, sparse search, graph expansion, and reranking then operate over those complementary representations.

The most important implementation choice is probably the semantic card. It should be concise, stable, and structured. A good card for each chunk might include: `purpose`, `entry_conditions`, `side_effects`, `calls`, `reads`, `writes`, `remotes`, `services`, `ui_paths`, `constants`, `security_checks`, `inferred_roles`, and `confidence`. For decompiled code, the card should explicitly state uncertainty, for example "possibly validates purchase amount" rather than "validates purchase amount" when evidence is weak. This gives embeddings natural-language surface area without poisoning the codebase with invented facts.

The second most important implementation choice is graph expansion. Many real questions are not answered by a single vector-nearest function. A helper may compute a price, its caller may validate the player, and a remote handler may be two edges away. Graph expansion should be bounded and explainable: from a top hit, include parent script, immediate callers, immediate callees, required module exports, and shared remote/event users, with strict token caps.

LIMITATIONS & CAVEATS

The decompiler literature primarily studies C/C++ binaries, not Roblox Luau bytecode or script decompilers [1], [2], [3], [5]. The principles transfer, but exact performance numbers do not. The reported name-recovery rates should be treated as evidence that context is useful, not as expected accuracy for this MCP server.

Many code embedding models are trained on clean public source code with comments and developer identifiers [7], [9], [10]. Decompiled Luau may be out of distribution. That increases the value of normalization, generated summaries, and hybrid search, but it also means the system needs local evaluation before making strong claims.

Generated summaries can introduce false semantics. This is manageable if summaries are annotations with confidence and evidence, but risky if the system rewrites code or hides raw source. For reverse-engineering workflows, provenance matters. The result card should always keep a path back to the exact raw lines.

HyDE and LLM reranking can improve recall and precision, but they add latency and cost [21], [22]. The default MCP tool should probably make them optional or use a two-tier mode: fast hybrid retrieval first, deeper expansion/reranking when the user asks for broader semantic search.

RECOMMENDATIONS

Implement semantic indexing as a staged pipeline:

1. Parse scripts into function, closure, callback, and fallback-window chunks. Preserve parent script, line range, Roblox instance path, enclosing table/module, and neighbor links [14], [15], [16].

2. Create a normalized-code field that replaces placeholder locals and generated argument names with stable placeholders, while preserving API names, global names, table keys, strings, constants, and Roblox path tokens [1], [8], [11].
3. Extract deterministic features before calling an LLM: services, remotes, requires, event connections, literals, table keys, property reads/writes, function calls, return shape, and obvious branch predicates [8], [12], [13].
4. Generate a short semantic card per chunk. Keep it structured, under a small token budget, and label role/name guesses as inferred. Store the prompt version and source hash so cards can be regenerated deterministically [2], [3], [22].
5. Store multiple searchable fields: raw code vector, normalized code vector, semantic-card vector, sparse/BM25 token index, and metadata/graph edges [7], [10], [17], [18].
6. Query with hybrid fusion. Retrieve from dense semantic-card, dense code, and sparse/BM25 branches, fuse with RRF or a simple weighted scheme, then expand top hits through bounded graph neighbors [18], [19].
7. Rerank only the candidate set. Use a code-aware reranker or an LLM scoring pass that sees the query, semantic card, metadata, and bounded raw lines. Deduplicate by function and script before returning context [19], [22].
8. Cache by chunk content hash. Index asynchronously, report full-index readiness, and avoid presenting partial indexes as complete semantic search [20].
9. Build a small eval set from actual decompiled scripts. Measure recall@k, MRR, answerability, token cost, duplicate rate, and latency. Run ablations before and after adding summaries, graph expansion, and HyDE [6], [19], [21].

The minimum viable version should be hybrid retrieval over deterministic extracted features plus generated semantic cards. The higher-quality version should add graph expansion and reranking. Training a decompiler-specific model is not the first step; the research suggests that strong gains are available from representation and retrieval design before custom model training.

COUNTEREVIDENCE REGISTER

Raw code embeddings are still useful. CodeBERT, UniXcoder, and Qdrant's code-search tutorial all support code embeddings as a viable retrieval component [7], [10], [14]. The recommendation is not to remove raw-code embeddings; it is to stop treating them as the only representation.

Identifier-aware models can benefit from meaningful developer-assigned identifiers [9]. In decompiled code, that signal is weaker, so identifier-aware models may underperform unless supported by normalization and generated annotations.

AST and parser-based chunking can fail on malformed decompiler output. That is why the recommended pipeline includes fallback recursive or sliding-window chunking rather than a parser-only approach [15],

[16].

Generated summaries can slow indexing and can be wrong [20], [22]. The mitigation is content-hash caching, asynchronous indexing, confidence labels, and always returning raw source evidence with the summary.

CLAIMS-EVIDENCE TABLE

Claim	Support
Decompiled identifiers are low-signal and often unrecoverable exactly.	Jaffe et al., DIRE, DIRTY, VarBERT, and STRIDE all describe lost names or decompiler placeholder names [1], [2], [3], [4], [5].
Structure and data-flow features should be promoted into the index.	GraphCodeBERT, code2vec, and code2seq show that structural representations add useful code semantics [8], [12], [13].
Function-level and AST-aware chunking are preferable to whole-file embedding.	CodeSearchNet, Qdrant, and LlamaIndex all center useful code retrieval around functions or AST/code-aware chunks [6], [14], [15].
Hybrid dense plus sparse retrieval is a better default than dense-only retrieval.	Elastic and Qdrant document hybrid retrieval and fusion, while Cody emphasizes complementary retrieval sources [17], [18], [19].
Generated summaries can bridge natural language to code, but they must be treated as annotations.	LanceDB's codebase RAG example uses generated comments and reranking, while decompiler papers caution that inferred names are contextual predictions [1], [3], [22].
Incremental indexing and evaluation are needed for practical MCP quality.	Cursor describes syntactic chunks, async embedding, and content-hash caching; CodeSearchNet and Cody emphasize evaluation for code retrieval [6], [19], [20].

METHODOLOGY APPENDIX

I used the deep-research workflow in standard mode. Scope was set around semantic indexing for decompiled code with low-quality identifiers. Retrieval targeted five source clusters: decompiled identifier recovery, code representation learning, semantic code-search benchmarks, code chunking/indexing implementation docs, and production retrieval architecture for coding assistants. Search CLI was unavailable locally, so retrieval used web search and direct source inspection.

The core evidence base contains 22 sources. Decompiled-code claims were triangulated across Jaffe et al., DIRE, DIRTY, VarBERT, and STRIDE. Code-representation claims were triangulated across CodeBERT, GraphCodeBERT, CodeT5, UniXcoder, code2vec, code2seq, and ContraCode. Retrieval architecture claims were triangulated across Qdrant, Elastic, Sourcegraph/Cody, Cursor, HyDE, and LanceDB. Recommendations are synthesized for an MCP semantic-search server and should be validated against actual decompiled scripts before changing production defaults.

BIBLIOGRAPHY

- [1] Jaffe, Lacomis, Schwartz, Le Goues, and Vasilescu (2018). "Meaningful Variable Names for Decompiled Code: A Machine Translation Approach". ICPC 2018.
<https://cmustrudel.github.io/papers/icpc18decompilation.pdf> (Retrieved: 2026-06-21)
- [2] Lacomis, Yin, Schwartz, Allamanis, Le Goues, Neubig, and Vasilescu (2019). "DIRE: A Neural Approach to Decompiled Identifier Naming". ASE 2019 / arXiv.
<https://arxiv.org/abs/1909.09029> (Retrieved: 2026-06-21)
- [3] Chen, Lacomis, Schwartz, Le Goues, Neubig, and Vasilescu (2022). "Augmenting Decompiler Output with Learned Variable Names and Types". USENIX Security 22.
<https://www.usenix.org/conference/usenixsecurity22/presentation/chen-qibin> (Retrieved: 2026-06-21)
- [4] Banerjee, Pal, Wang, and Baral (2021). "Variable Name Recovery in Decompiled Binary Code using Constrained Masked Language Modeling". arXiv. <https://arxiv.org/abs/2103.12801> (Retrieved: 2026-06-21)
- [5] Green, Schwartz, Le Goues, and Vasilescu (2024). "STRIDE: Simple Type Recognition In Decompiled Executables". arXiv. <https://arxiv.org/abs/2407.02733> (Retrieved: 2026-06-21)
- [6] GitHub and Microsoft Research (2019). "CodeSearchNet: Datasets, tools, and benchmarks for representation learning of code". GitHub. <https://github.com/github/CodeSearchNet> (Retrieved: 2026-06-21)
- [7] Feng et al. (2020). "CodeBERT: A Pre-Trained Model for Programming and Natural Languages". Findings of EMNLP / arXiv. <https://arxiv.org/abs/2002.08155> (Retrieved: 2026-06-21)
- [8] Guo et al. (2020). "GraphCodeBERT: Pre-training Code Representations with Data Flow". arXiv. <https://arxiv.org/abs/2009.08366> (Retrieved: 2026-06-21)
- [9] Wang, Wang, Joty, and Hoi (2021). "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation". EMNLP / arXiv.
<https://arxiv.org/abs/2109.00859> (Retrieved: 2026-06-21)
- [10] Guo, Lu, Duan, Wang, Zhou, and Yin (2022). "UniXcoder: Unified Cross-Modal Pre-training for Code Representation". ACL / arXiv. <https://arxiv.org/abs/2203.03850> (Retrieved: 2026-06-21)
- [11] Jain, Jain, Zhang, Abbeel, Gonzalez, and Stoica (2021). "Contrastive Code Representation Learning". EMNLP / arXiv. <https://arxiv.org/abs/2007.04973> (Retrieved: 2026-06-21)

- [12]** Alon, Zilberstein, Levy, and Yahav (2019). "code2vec: Learning Distributed Representations of Code". POPL / arXiv. <https://arxiv.org/abs/1803.09473> (Retrieved: 2026-06-21)
- [13]** Alon, Brody, Levy, and Yahav (2019). "code2seq: Generating Sequences from Structured Representations of Code". ICLR / arXiv. <https://arxiv.org/abs/1808.01400> (Retrieved: 2026-06-21)
- [14]** Qdrant (n.d.). "Semantic Search for Code". Qdrant Documentation. <https://qdrant.tech/documentation/tutorials-develop/code-search/> (Retrieved: 2026-06-21)
- [15]** LlamaIndex (n.d.). "Node Parsers / Text Splitters: CodeSplitter". LlamaIndex Documentation. https://developers.llamaindex.ai/typescript/framework/modules/data/ingestion_pipeline/transformations/node-parser/ (Retrieved: 2026-06-21)
- [16]** LangChain (n.d.). "Splitting recursively". LangChain Documentation. https://docs.langchain.com/oss/python/integrations/splitters/recursive_text_splitter (Retrieved: 2026-06-21)
- [17]** Elastic (n.d.). "What is hybrid search? How it works and when to use it". Elastic. <https://www.elastic.co/what-is/hybrid-search> (Retrieved: 2026-06-21)
- [18]** Qdrant (n.d.). "Hybrid Queries". Qdrant Documentation. <https://qdrant.tech/documentation/search/hybrid-queries/> (Retrieved: 2026-06-21)
- [19]** Hartman et al. (2024). "AI-assisted Coding with Cody: Lessons from Context Retrieval and Evaluation for Code Recommendations". RecSys 2024 / arXiv. <https://arxiv.org/abs/2408.05344> (Retrieved: 2026-06-21)
- [20]** Cursor (2026). "Securely indexing large codebases". Cursor Blog. <https://cursor.com/blog/secure-codebase-indexing> (Retrieved: 2026-06-21)
- [21]** Gao, Ma, Lin, and Callan (2022). "Precise Zero-Shot Dense Retrieval without Relevance Labels". arXiv. <https://arxiv.org/abs/2212.10496> (Retrieved: 2026-06-21)
- [22]** Shubham (2024). "Building RAG on codebases: Part 2". LanceDB Blog. <https://www.lancedb.com/blog/building-rag-on-codebases-part-2> (Retrieved: 2026-06-21)